End2Race: An End-to-End Learning Framework for Multi-Vehicle Autonomous Racing

Anonymous Authors

***Abstract*— Autonomous racing serves as a compelling testbed for advancing autonomous vehicle systems. F1Tenth, a 1/10-scale vehicular platform, has been widely adopted for autonomous racing education and research. Annual competitions held worldwide bring teams together to compete in intense, head-to-head racing. Yet, leading solutions in these tournaments remain dominated by rule-based approaches. While learning-based methods have been proposed, they focus primarily on single-vehicle settings, leaving challenging multi-vehicle interactions largely unexplored. To address this gap, we introduce End2Race, an end-to-end learning framework specifically designed for head-to-head autonomous racing. Using this framework, we develop a computationally efficient policy network that achieves an inference time below 1 ms on an F1Tenth onboard computer. Extensive evaluations show that the policy generalizes effectively to novel tracks, demonstrating high racing speeds, advanced overtaking capabilities, and robust safety. These results significantly surpass established rule-based baselines. The codebase will be made publicly available.**

I. Introduction

Autonomous racing combines perception, planning, and control under high-speed, interactive, and computationally constrained conditions. Enhancing algorithmic capabilities in this domain is therefore of interest to the broader development of autonomous driving. F1Tenth [1], a 1/10-scale vehicular platform, provides an accessible testbed for autonomous racing education and research. Equipped with a 2D LiDAR or an RGB-D camera, the vehicle can operate at speeds up to 17.9 m/s. Annual competitions held worldwide attract broad participation from the community.

Existing approaches for autonomous racing can be broadly grouped into map-based, reactive, and learning-based methods. Among them, map-based pipelines are the most widely used, relying on a pre-constructed map to follow a globally optimized raceline. Prior to racing, the vehicle is manually driven to map the track layout using SLAM [2]. A minimum-curvature raceline is then computed to serve as the global reference path [3]. During racing, a particle filter [4] tracks the vehicle's position, enabling a lattice planner [5] to evaluate feasible local trajectories. Motion controllers such as Pure Pursuit [6] or model predictive control (MPC) [7] then execute the selected trajectory. However, map-based methods have two limitations. First, they require preprocessing and cannot operate on unseen tracks. Second, the multi-stage pipeline incurs a runtime delay of approximately 30 ms on onboard hardware [8], [9]. Combined with sensor latency, the vehicle can travel up to half a meter before updated control commands take effect. This delay is non-critical under most conditions, but becomes consequential when a high-speed straight segment leads directly into a sharp curve, leaving the vehicle susceptible to boundary collisions. Such collisions are widely observed in F1Tenth competitions.

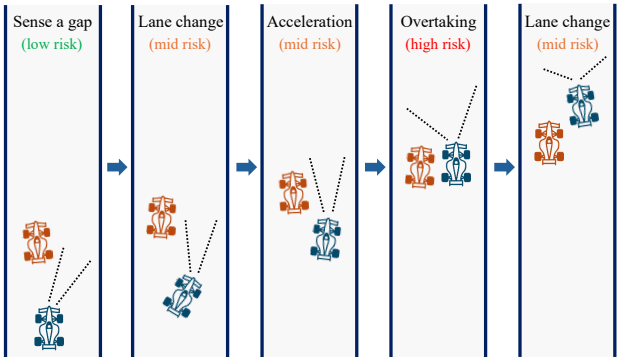

Fig. 1. Illustration of the five-stage overtaking process.

Reactive methods provide a faster, rule-based alternative. They operate directly on current sensor measurements and do not require map information, enabling operation on unknown tracks. The Follow-the-Gap algorithm [10], for example, identifies the largest free region and steers toward its midpoint. However, these methods operate instantaneously on isolated observations with limited temporal context, leading to planning inconsistencies and control instability [11]. As a result, they are rarely used in F1Tenth competitions.

Beyond rule-based methods, learning-based approaches have been widely studied in F1Tenth. Early work focused on single-vehicle navigation [12], [13], whereas recent studies have shifted toward multi-vehicle interactions. Nevertheless, existing approaches face severe limitations, such as training on static obstacles and testing directly against dynamic opponents [14], relying on a small set of human demonstrations [15], or operating at impractically low racing speeds [16]. Importantly, none of these methods explicitly tackles diverse, interactive overtaking scenarios, which are intrinsically challenging and represent a key factor in the race outcome. We illustrate a simplified five-stage overtaking process in Fig. 1. First, the ego vehicle identifies an opening, executes a lane change to align with it, and

Funding information omitted for double-blind review.
[1]Affiliation omitted for double-blind review.
[2]Affiliation omitted for double-blind review.
[3]Affiliation omitted for double-blind review.
[3]Affiliation omitted for double-blind review.
[3]Affiliation omitted for double-blind review.
[3]Affiliation omitted for double-blind review.
†Equal contribution.
*Corresponding author information omitted.

accelerates rapidly to catch up with the opponent. During the pass, the vehicles travel side by side for an extended period with minimal safety margins. After completing the pass, the ego vehicle returns to the track center to regain lateral clearance. Successful overtaking therefore depends on continuous decision-making, precise maneuver execution, rapid adaptation to the opponent, and minimal response time. This requires sufficient model capacity and low inference latency, two conflicting demands that existing methods fail to reconcile.

To address these limitations, we propose **End2Race**, an end-to-end learning framework for real-time, head-to-head autonomous racing. The framework supports the automatic generation of overtaking scenarios under diverse conditions, enabling systematic policy training and evaluation at scale. Utilizing this framework, we train a racing policy based on a Gated Recurrent Unit (GRU) architecture [17], which takes a 360-degree 2D LiDAR scan and the current ego speed as inputs and directly outputs steering and speed commands. The policy is lightweight and efficient, achieving an inference time below 1 ms on an F1Tenth onboard computer. Training follows a two-stage pipeline that initializes the policy through imitation learning and then fine-tunes it using reinforcement learning. Across training and test tracks, the resulting policy demonstrates high racing speeds, strong safety performance, and competitive overtaking skills.

The main contributions of this work are as follows:

1) We introduce End2Race, an end-to-end learning framework for autonomous racing that supports the automatic generation of overtaking scenarios, enabling policy training and evaluation at scale.
2) We design a computationally efficient recurrent policy network with a sensing-to-control latency below 1 ms, supporting real-time operation at high racing speeds.
3) We demonstrate that our learned policy generalizes effectively to novel tracks, achieving high racing speeds, robust safety, and competitive overtaking maneuvers.

II. Related Work

A. Imitation Learning

Imitation learning (IL) trains a driving policy from expert demonstrations. Sun et al. [12] generated training data using Pure Pursuit and evaluated Behavioral Cloning (BC) [18], DAgger [19], HG-DAgger [20], and EIL [21]. They found that all policies completed the training track but failed to generalize to unseen tracks. Paluch et al. [22] trained a feedforward policy to imitate nonlinear MPC along a raceline, achieving faster lap times than Pure Pursuit. TinyLidarNet [14] trained a compact convolutional network to avoid static obstacles and evaluated it against dynamic opponents. Zhang and Loidl [15] trained driving policies from human overtaking demonstrations, but relied on limited training and evaluation scenarios.

B. Model-Free Reinforcement Learning

Model-free RL optimizes control policies through environmental interaction to maximize cumulative reward. Hamilton et al. [23] compared DDPG [24], SAC [25], and BC [18], finding that RL policies achieved faster lap times but incurred higher collision rates. Steiner et al. [16] trained a TD3 [26] policy for head-to-head racing, demonstrating that opponent-inclusive training improves overtaking success. However, their deep feedforward policy network introduced substantial inference delays, which compromised real-time responsiveness and forced operating speeds below 2 m/s.

C. Model-Based Reinforcement Learning

Model-based RL learns a predictive model of the environment and uses it to optimize control policies through imagined rollouts. For instance, Dreamer [27] learns a recurrent state-space model and trains actor-critic networks entirely within a latent space. Brunnbauer et al. [13] applied Dreamer to autonomous racing, reporting better sample efficiency and lap completion than model-free baselines.

D. Hybrid Learning Methods

Hybrid methods integrate learning-based modules into classical planning or control pipelines. For example, Dwivedi et al. [28] combined high-level path guidance from a global planner with a low-level learned tracking controller. To guide overtaking decisions, Bhargav et al. [29] modeled overtaking success probabilities across track segments to dynamically select lateral planning modes. Similarly, Trumpp et al. [30] used an artificial potential field planner for baseline obstacle avoidance and trained a residual RL policy to provide corrective control for overtaking.

III. Methodology

We develop End2Race using the F1TENTH simulator, which provides high-fidelity vehicle dynamics and accurate 2D LiDAR simulation. Four tracks are selected: Austin, Hockenheim, MoscowRaceway, and Nuerburgring, each reflecting a real-world circuit layout. Among these, Austin is used for training, while the rest are held out for evaluation. For each track, an automated workflow generates diverse overtaking scenarios that pair an ego vehicle with a leading opponent. Using demonstrations from a map-based expert, we first train a GRU-based network through BC, and then fine-tune the resulting policy with Proximal Policy Optimization (PPO) [31]. The following sections describe each stage in detail (Fig. 2).

A. Overtaking Scenario Setup

Each overtaking scenario pairs an ego vehicle with an opponent initially positioned ahead of it and has a duration of 8 seconds. Vehicle placement is expressed in Frenet coordinates, in which the longitudinal direction measures progress around the circuit and the lateral direction denotes position across the track width. We specify each scenario using four parameters, $(s_0, \Delta s_0, r_{\text{opp}}, \alpha_{\text{opp}})$.

1) Ego Starting Position (s_0): For each track, we define 80 ego starting positions equally spaced along the circuit. On the 420 m Austin track, for example, adjacent starting positions are separated by approximately 5.2 m.

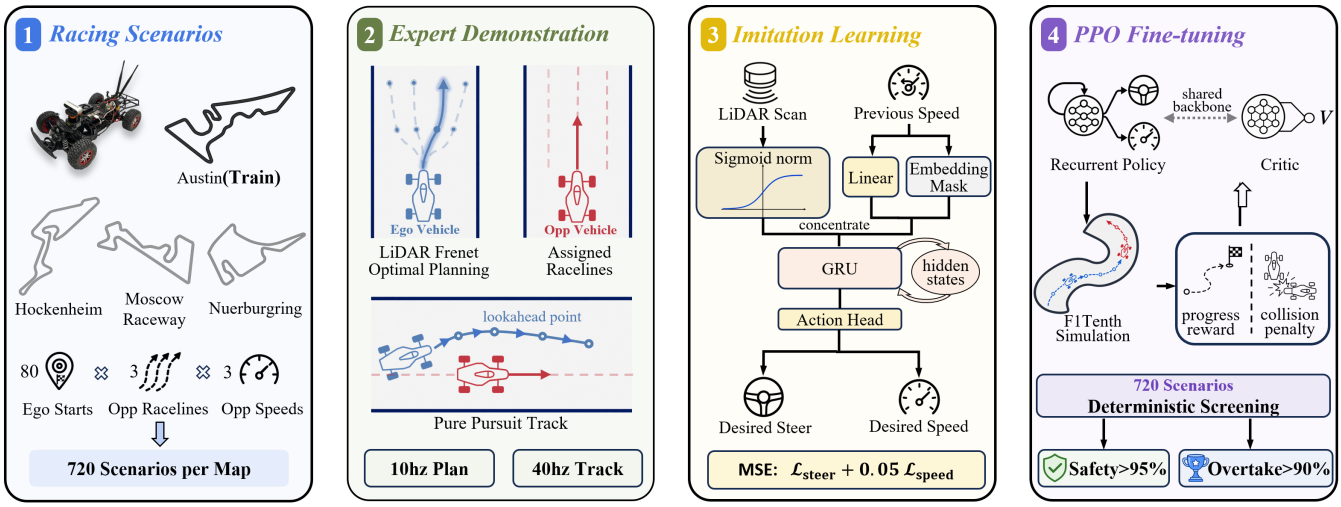

Fig. 2. End2Race system architecture.

2) Longitudinal Gap (Δs_0): Here, Δs_0 specifies how far the opponent is positioned ahead of the ego vehicle at the start of a scenario. A small gap provides insufficient starting space for the ego vehicle to maneuver, while a large gap requires the ego vehicle to spend more time catching the opponent, leaving less time for interaction. We therefore set $\Delta s_0 = 3$ m to balance these considerations.

3) Opponent Raceline (r_{opp}): Beyond longitudinal placement, the lateral positions of the ego vehicle and opponent are critical to the overtaking outcome. We represent this configuration using the opponent raceline r_{opp}, which assigns the opponent to one of three reference paths. Following the framework of Heilmeier et al. [3], we define left, center, and right paths positioned at 35%, 50%, and 65% across the track width from the left boundary. On the Austin circuit, this yields approximately 0.25 m of lateral separation between the center and each side raceline. An opponent on the center raceline restricts clearance on both sides, whereas one on a side raceline leaves a wider passing corridor.

4) Opponent Speed Scale (α_{opp}): Each reference raceline provides an optimized speed profile based on track curvature and vehicle dynamics. Here, we set a maximum speed of 7.5 m/s, as empirical testing showed that higher values risk compromising control stability. Note that if the opponent follows this profile directly, an ego vehicle driving at the same speed cannot catch up or overtake. To create overtaking opportunities, we multiply the opponent's reference speed by a factor $\alpha_{\text{opp}} \in \{0.4, 0.6, 0.8\}$. Values below 0.4 make the opponent unrealistically slow, while those above 0.8 often prevent the ego vehicle from catching up in time.

Combining 80 ego starting positions, a fixed longitudinal gap, three opponent racelines, and three opponent speed scales yields 720 scenarios per track. During each 8-second scenario, the opponent passively tracks its assigned raceline using Pure Pursuit. It does not react to the ego vehicle, nor does it engage in any adversarial behavior, such as defensive blocking or erratic speed changes [32].

B. Expert Demonstration

We use a map-based lattice planner to generate overtaking demonstrations for the ego vehicle. Here, the ego uses the center raceline as its global reference path and operates under the full speed profile. The lattice planner, adapted from the Frenet optimal trajectory framework [33], generates local candidate trajectories across a range of lateral offsets and terminal speeds relative to this reference. This allows the ego vehicle to temporarily deviate from the centerline to avoid obstacles and execute overtaking maneuvers.

At each timestep, the lattice planner generates a set of candidate trajectories around the reference path, discards those that violate vehicle dynamic constraints, and selects the trajectory τ that minimizes the cost function $\mathcal{J}(\tau)$:

$$\mathcal{J}(\tau) = \frac{1}{N}\sum_{i=1}^{N}\left(1 - \frac{v_i}{v_{\max}}\right) + \lambda_c \max_i \left[\max\left(0, 1 - \frac{d_i}{d_c}\right)\right]^2 \tag{1}$$

where $N = 6$ is the number of future trajectory points sampled at 0.1 s intervals; v_i is the projected vehicle speed at point i; $v_{\max} = 7.5$ m/s is the speed limit; d_i is the distance from point i to the nearest obstacle detected by the LiDAR scan; and $d_c = 0.5$ m is the clearance threshold. The first term encourages the vehicle to travel at higher speeds up to the speed limit, while the second term penalizes proximity to obstacles. They are balanced by the collision weight $\lambda_c = 5$, determined empirically. The selected local trajectory is then tracked using a Pure Pursuit controller.

C. Training Dataset

We collect 720 eight-second expert demonstrations on the Austin track. Training data is recorded at 40 Hz to match the onboard LiDAR frequency, yielding 320 observation-action pairs per scenario. Of these, 620 result in successful overtakes, 23 in car-following, and 77 in collisions. Notably, these collisions stem primarily from the lattice planner's forward-only obstacle avoidance, which does not account for a moving opponent alongside the ego vehicle. Consequently,

Fig. 3. Expert-planner failure during overtaking, shown in temporal order from (a) to (c). Blue denotes the ego vehicle and its planned trajectory, and red denotes the opponent. In this scenario, the opponent follows the right raceline with a speed scale of 0.8.

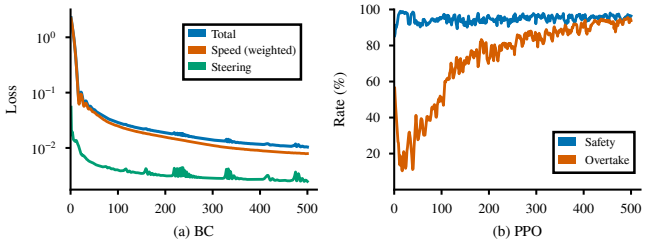

Fig. 4. Training curves for the two-stage policy optimization, with each stage trained for 500 epochs. (a) BC loss, with the speed loss weighted by 0.05. (b) PPO safety and overtake rates on the 720 Austin scenarios.

the ego vehicle may return to the center raceline before it has fully cleared the opponent, causing a sideswipe collision. This risk increases as the speed difference between the two vehicles decreases, with most failures occurring at an opponent speed scale of 0.8. Figure 3 illustrates one such failure, where the ego prematurely returns to the centerline during the overtake and collides with the opponent.

These collisions can be mitigated with explicit opponent detection, trajectory prediction, and interaction modeling, as shown in Predictive Spliner [34]. However, such approaches remain fundamentally constrained by hand-crafted heuristics and rigid safety margins that struggle to accommodate different scenarios. We instead exclude the collision demonstrations from BC training and rely on downstream PPO fine-tuning to learn safe interactive behavior (Sec. III-E).

D. Model Architecture

We use a GRU-based network to capture temporal dependencies and map sequential observations to vehicle control commands. At each timestep, the observation comprises a 360-degree 2D LiDAR scan of 180 beams and the current ego speed. The control output specifies the desired speed and steering angle. The following introduces key design details, including LiDAR normalization, ego-speed projection, and the network design.

1) LiDAR Normalization: Bufacchi and Iannetti show that human interaction with surrounding space is proximity-dependent, with behavioral responses strengthening progressively as object distance decreases [35]. The same principle can be applied to autonomous racing, where the influence of surrounding obstacles on ego motion similarly depends on their distance. To emulate this aspect of human spatial cognition, we transform each of the 180 LiDAR beams using a sigmoid function that maps the raw scan value, $x \in [0, 10]$, to a normalized value, $\sigma(x) \in [0, 1]$:

$$\sigma(x) = \left(-\frac{1}{1 + e^{-kx}} + 1 \right) \cdot 2 \tag{2}$$

Here, the parameter k controls the effective sensing range and is set to 0.3. Under this transformation, the normalized scan value increases nonlinearly as obstacles approach and asymptotically decays to zero with distance.

2) Ego-Speed Projection: To maintain dynamic consistency, we incorporate the current ego speed as a policy input, projecting the scalar speed v_t through a linear layer into a 30-dimensional vector $\psi(v_t)$. To prevent shortcut learning [36], in which the model simply copies the current speed into its prediction, we randomly mask the projected speed with a learnable embedding $\mathbf{e}_{\text{mask}}$ as follows:

$$\widetilde{\psi}(v_t) = \begin{cases} \psi(v_t), & \text{with probability } 1 - p, \\ \mathbf{e}_{\text{mask}}, & \text{with probability } p, \end{cases} \tag{3}$$

Here, we set $p = 0.2$, encouraging the model to determine desired speed from LiDAR observations when the current speed is masked and to incorporate it when available.

3) Network Design: The 180-dimensional normalized LiDAR scan and the 30-dimensional speed projection are concatenated to form a 210-dimensional input vector. This input is passed to a single-layer GRU network, which maintains a 420-dimensional hidden state and updates it at each timestep. This hidden state is then passed to an MLP action head to produce the desired speed and steering angle. In total, the model has 850,556 trainable parameters.

E. Policy Optimization

Training consists of two stages: we first use BC to learn a policy from expert demonstrations, and then fine-tune it with PPO via direct environment interaction.

1) Behavioral Cloning: Each 8-second scenario is processed as a whole, generating a control prediction at every timestep and aggregating the loss across the full sequence. The loss function combines the mean squared errors (MSE) of speed (m/s) and steering angle (rad), weighting the speed loss by 0.05 to balance their scales:

$$\mathcal{L} = 0.05\,\mathcal{L}_{\text{speed}} + \mathcal{L}_{\text{steer}}. \tag{4}$$

2) Proximal Policy Optimization: The policy is fine-tuned across the 720 scenarios on the Austin track. In each training iteration, we collect one trajectory per scenario, combining them into a single batch to perform a policy update. At each timestep t, the reward function r_t combines a raceline-progress term with a collision penalty:

$$r_t = 0.01 d_t - 3 c_t, \tag{5}$$

where d_t denotes the distance traveled along the raceline, and c_t is a binary collision indicator. Together, these terms encourage the ego vehicle to pursue high racing speeds while prioritizing safety during overtaking maneuvers.

TABLE I

BENCHMARK EVALUATION: SINGLE-VEHICLE AND HEAD-TO-HEAD RACING PERFORMANCE

Setting	Metric	Austin (Train)			Hockenheim			MoscowRaceway			Nuerburgring		
		Expert	BC	PPO	Expert	BC	PPO	Expert	BC	PPO	Expert	BC	PPO
Single-Vehicle	Mean Speed (m/s)	6.7	6.7	**8.7**	6.8	6.9	**8.8**	6.5	6.8	**8.5**	6.9	7.0	**8.9**
	Mean Lap Time (s)	63.2	62.7	**48.7**	53.2	52.5	**41.1**	50.6	47.7	**38.5**	64.7	63.7	**50.2**
Head-to-Head	Safety Rate (%)	89.3	82.8	**98.1**	91.8	78.6	**97.5**	87.2	76.7	**97.1**	92.8	81.5	**97.9**
	Overtake Rate (%)	86.1	62.8	**96.1**	90.6	66.1	**97.1**	83.5	69.0	**96.8**	92.4	74.6	**97.5**

IV. EXPERIMENTS

We evaluate End2Race across the training and three test tracks. The evaluation includes both single-vehicle timed trials and head-to-head scenarios. Visual illustrations further demonstrate representative policy behaviors.

A. Training Details

The BC policy is trained for 500 epochs using the Adam optimizer with a learning rate of 10^{-4}. Training completes in five minutes on a single L40S GPU. The loss decreases rapidly during early epochs and then gradually converges (Fig. 4(a)). Additional training would further reduce loss, but risk overfitting and yield no performance gains.

PPO starts from the BC checkpoint with a newly initialized value head. Training runs for 500 epochs with a learning rate of 10^{-5} and a clipping range of $[0.8, 1.2]$. Since online simulation is time-consuming, we use sixty-four CPU cores and four L40S GPUs, requiring approximately 16 hours in total. During early epochs, the safety rate rises sharply while the overtake rate drops, reflecting conservative driving. As training progresses, the overtake rate steadily increases while maintaining high safety, demonstrating the acquisition of adaptive driving skills (Fig. 4(b)).

B. Single-Vehicle Timed Trials

We first evaluate End2Race in single-vehicle timed trials to test whether the policy can complete the track at high speeds. Table I reports the mean speed and lap time of three methods: the map-based expert, BC, and PPO, all of which complete the evaluation without collision. Compared to the expert, which relies on explicit track knowledge and an optimized reference raceline, the map-free BC policy achieves comparable mean speeds and lap times across all four tracks, demonstrating effective learning and generalization.

The PPO policy achieves significantly higher mean speeds and shorter lap times on every track. These gains arise from continued interactive exploration, through which the policy discovers racing strategies that often resemble techniques used by human racers. One example is corner-exit track-out, in which the vehicle moves from the inside to the outside of a curve to reduce speed loss. Another example is corner-exit acceleration and corner-entry braking, in which the vehicle accelerates immediately upon exiting a curve, maintains high speed along the following straight, and brakes sharply just before entering the next curve [37].

C. Head-to-Head Racing

We evaluate head-to-head performance using the overtake and safety rates. The former is defined as the proportion of scenarios ending in a successful overtake, while the latter includes both successful overtakes and collision-free car-following outcomes. The map-based expert achieves a safety rate of approximately 90% across all tracks, with the majority being successful overtakes (Table I).

Compared to the expert, the BC policy adopts a more conservative racing strategy, achieving lower overtake rates. However, this conservatism does not translate into improved safety. We attribute this outcome to the imitation learning objective itself: while the network can mimic demonstrated control commands, it does not establish the internal representations and tactical reasoning essential for vehicle interactions. Consequently, the policy remains effective for single-vehicle trials but struggles under more complex conditions.

This limitation is resolved during the PPO stage, which substantially outperforms the expert in both safety and overtake rates. This demonstrates that PPO not only bridges the performance gap, but also overcomes the limitations of the expert demonstration data.

D. Scenario Illustration

Figure 5 illustrates four cases that highlight the model's behavior during overtaking encounters. All examples are rollouts generated by the PPO policy on the Austin circuit.

Overtaking: Fig. 5(1) illustrates an overtaking sequence along a straight segment. Upon observing a slower opponent ahead, the ego vehicle initiates a lateral shift while sustaining cruising speed. It maintains a stable lateral clearance alongside the opponent, then smoothly returns to the track center without an abrupt cut-back.

Fig. 5(2) depicts a cornering overtake. The ego vehicle initially follows the opponent, preparing to pass. When an unexpected sharp curve emerges, it applies moderate braking to sustain higher speed through the turn, then accelerates early at corner exit to complete the pass.

Car-Following: Fig. 5(3) shows a car-following case. During an overtaking maneuver, the ego vehicle is blocked at the inner corridor by the opponent, forcing it to brake heavily and almost come to a complete stop. Once the opponent proceeds, the ego vehicle accelerates to resume driving.

Collision: Fig. 5(4) captures a failure case. As the vehicles travel side by side, the track suddenly narrows. Left with no

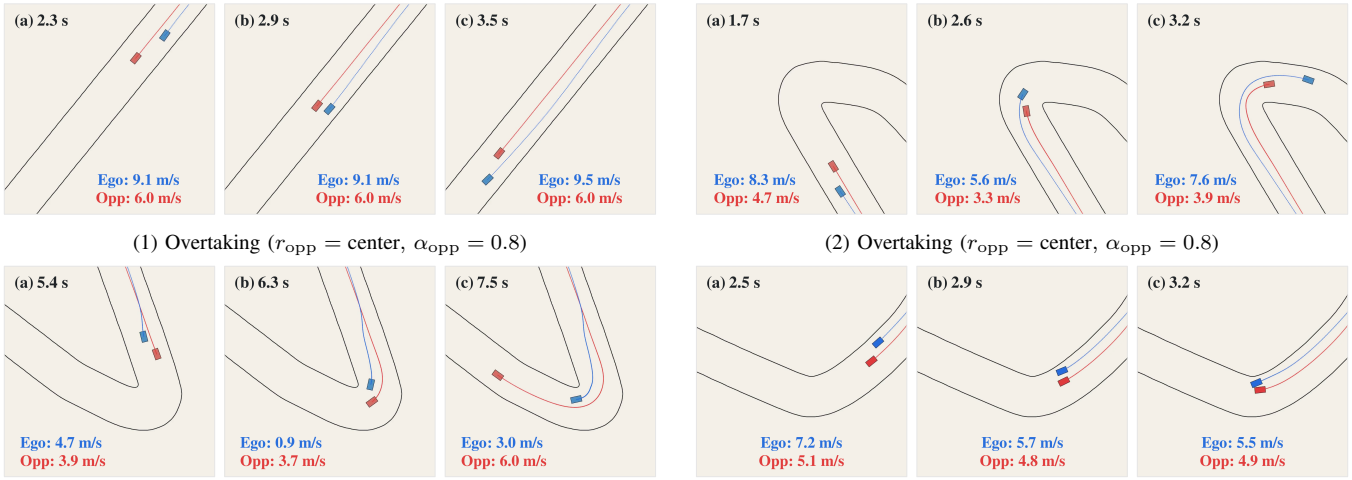

Fig. 5. Representative head-to-head interaction scenarios across time on the Austin circuit. Blue denotes the ego vehicle and red denotes the opponent, with historical trajectories rendered in matching colors. Each frame indicates the elapsed time and vehicle speeds.

room along the boundary, the ego vehicle steers toward the opponent, resulting in a collision.

Together, these cases demonstrate the tactical racing and overtaking capabilities of the learned policy. Nevertheless, as the policy operates without a map, it cannot predict forward track geometry, which can lead to unfavorable conditions under sudden track constrictions.

E. Ablation Study

We conduct an ablation study to examine the effect of design choices in our model architecture, covering two key components: LiDAR normalization and temporal sequence modeling (Table II). All variants are trained using behavioral cloning and evaluated on the Austin circuit.

1) LiDAR Normalization: We compare the proposed sigmoid normalization with two alternatives: no normalization and linear normalization, all sharing the same GRU architecture. The unnormalized model fails to complete a single-vehicle lap. Linear normalization provides a viable policy, but leaves a large performance gap in head-to-head racing. This shows that the sigmoid formulation is crucial in our design for effective driving.

2) Temporal Information: To assess the importance of temporal information, we evaluate an MLP that operates only on the latest observation. The MLP replaces the GRU with a $210 \to 420$ linear layer, followed by the action head. The resulting policy exhibits heuristic driving behavior, achieving a higher vehicle speed and overtake rate at the cost of a substantial drop in safety. These results confirm that temporal cues are essential for continuous interaction modeling.

3) Temporal Modeling: We evaluate a Transformer [38] as an alternative temporal model. The model operates on a maximum history sequence of 20 tokens, which corresponds to a span of 0.5 s. Each token represents one timestep and has a dimension of 210. The tokens are processed through two encoder layers with bidirectional self-attention, three attention heads, and a feedforward dimension of 420. The output of the latest token feeds into the action head to produce the control commands. The resulting policy achieves a higher overtake rate, but with a lower safety rate. Additionally, the Transformer incurs four times the inference latency of the GRU. To balance overtaking, safety, and real-time responsiveness, we retain the GRU as our default architecture. Meanwhile, we recognize the Transformer as a practical alternative.

TABLE II
ABLATION STUDY ON THE AUSTIN CIRCUIT

Variant	Single-Vehicle		Head-to-Head	
	Speed (m/s)	Lap Time (s)	Overtake (%)	Safety (%)
No Norm.	–	–	16.3	21.1
Linear Norm.	6.4	64.9	47.0	73.3
MLP	**6.9**	**60.9**	67.6	70.4
Transformer	6.6	63.8	**73.1**	80.6
GRU (BC)	6.7	62.7	62.8	**82.8**

V. CONCLUSION

In this work, we introduce End2Race, an end-to-end learning framework for head-to-head autonomous racing. Our proposed policy achieves low inference latency and delivers competitive racing performance. Future work will focus on deploying the policy to a physical vehicle and participating in F1Tenth competitions.

REFERENCES

[1] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1tenth: An open-source evaluation environment for continuous control and reinforcement learning," *Proceedings of Machine Learning Research*, vol. 123, 2020.

[2] S. Macenski and I. Jambrecic, "Slam toolbox: Slam for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

[3] A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, "Minimum curvature trajectory planning and control for an autonomous race car," *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497–1527, 2020.

[4] C. Walsh and S. Karaman, "Cddt: Fast approximate 2d ray casting for accelerated localization," *arXiv preprint arXiv:1705.01167*, 2017.

[5] M. Pivtoraiko, R. A. Knepper, and A. Kelly, "Differentially constrained mobile robot motion planning in state lattices," *Journal of Field Robotics*, vol. 26, pp. 308 – 333, March 2009.

[6] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Tech. Rep. CMU-RI-TR-92-01, Carnegie Mellon University, Pittsburgh, PA, January 1992.

[7] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Transactions on Control Systems Technology*, vol. 15, no. 3, pp. 566–580, 2007.

[8] Z. Zang, H. Zheng, J. Betz, and R. Mangharam, "Local_INN: Implicit map representation and localization with invertible neural networks," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11742–11748, 2023.

[9] F. Muzzini, N. Capodieci, F. Ramanzin, and P. Burgio, "Gpu implementation of the frenet path planner for embedded autonomous systems: A case study in the F1TENTH scenario," *Journal of Systems Architecture*, vol. 154, p. 103239, 2024.

[10] V. Sezer and M. Gokasan, "A novel obstacle avoidance algorithm: "follow the gap method"," *Robotics and Autonomous Systems*, vol. 60, no. 9, pp. 1123–1134, 2012.

[11] T. Hossain, H. Habibullah, R. Islam, and R. V. Padilla, "Local path planning for autonomous mobile robots by integrating modified dynamic-window approach and improved follow the gap method," *Journal of Field Robotics*, vol. 39, no. 4, pp. 371–386, 2022.

[12] X. Sun, M. Zhou, Z. Zhuang, S. Yang, J. Betz, and R. Mangharam, "A benchmark comparison of imitation learning-based control policies for autonomous racing," in *2023 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1–5, 2023.

[13] A. Brunnbauer, L. Berducci, A. Brandstátter, M. Lechner, R. Hasani, D. Rus, and R. Grosu, "Latent imagination facilitates zero-shot transfer in autonomous racing," in *2022 International Conference on Robotics and Automation (ICRA)*, pp. 7513–7520, 2022.

[14] M. M. Zarrar, Q. Weng, B. Yerjan, A. Soyyigit, and H. Yun, "Tinyl-idarnet: 2d lidar-based end-to-end deep learning model for f1tenth autonomous racing," 2024.

[15] J. Zhang and H. Loidl, "F1tenth: An over-taking algorithm using machine learning," in *2023 28th International Conference on Automation and Computing (ICAC)*, pp. 01–06, 2023.

[16] E. Steiner, D. van der Spuy, F. Zhou, A. Pama, M. Liarokapis, and H. Williams, "Accelerating real-world overtaking in f1tenth racing employing reinforcement learning methods," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4528–4534, IEEE, 2025.

[17] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.

[18] C. Sammut, "Behavioral cloning," 01 2011.

[19] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pp. 627–635, JMLR Workshop and Conference Proceedings, 2011.

[20] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "Hg-dagger: Interactive imitation learning with human experts," in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 8077–8083, IEEE, 2019.

[21] J. Spencer, S. Choudhury, M. Barnes, M. Schmittle, M. Chiang, P. Ramadge, and S. Srinivasa, "Expert intervention learning: An online framework for robot learning from explicit and implicit human feedback," *Autonomous Robots*, vol. 46, no. 1, pp. 99–113, 2022.

[22] M. Paluch, F. Bolli, X. Deng, A. Rios Navarro, C. Gao, and T. Delbruck, "Fpga hardware neural control of cartpole and f1tenth race car," 2024.

[23] N. Hamilton, P. Musau, D. M. Lopez, and T. T. Johnson, "Zero-shot policy transfer in autonomous racing: Reinforcement learning vs imitation learning," in *2022 IEEE International Conference on Assured Autonomy (ICAA)*, pp. 11–20, 2022.

[24] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations*, 2016.

[25] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*, pp. 1861–1870, Pmlr, 2018.

[26] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*, pp. 1587–1596, PMLR, 2018.

[27] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," *arXiv preprint arXiv:1912.01603*, 2019.

[28] T. Dwivedi, T. Betz, F. Sauerbeck, P. Manivannan, and M. Lienkamp, "Continuous control of autonomous vehicles using plan-assisted deep reinforcement learning," in *2022 22nd International Conference on Control, Automation and Systems (ICCAS)*, pp. 244–250, 2022.

[29] J. Bhargav, J. Betz, H. Zheng, and R. Mangharam, "Track based offline policy learning for overtaking maneuvers with autonomous racecars," 2021.

[30] R. Trumpp, E. Javanmardi, J. Nakazato, M. Tsukada, and M. Caccamo, "Racemop: Mapless online path planning for multi-agent autonomous racing using residual policy learning," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8449–8456, 2024.

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[32] H. Zheng, Z. Zhuang, J. Betz, and R. Mangharam, "Game-theoretic objective space planning," 2023.

[33] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, "PythonRobotics: A python code collection of robotics algorithms," 2018.

[34] N. Baumann, E. Ghignone, C. Hu, B. Hildisch, T. Hämmerle, A. Bettoni, A. Carron, L. Xie, and M. Magno, "Predictive spliner: Data-driven overtaking in autonomous racing using opponent trajectory prediction," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1816–1823, 2025.

[35] R. J. Bufacchi and G. D. Iannetti, "An action field theory of peripersonal space," *Trends in Cognitive Sciences*, vol. 22, no. 12, pp. 1076–1090, 2018.

[36] R. Geirhos, J. Jörn-Henrik, C. Michaelis, R. Zemel, B. Wieland, B. Matthias, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, 11 2020.

[37] W. C. Mitchell, R. Schroer, and D. B. Grisez, "Driving the traction circle," Tech. Rep. 2004-01-3545, SAE International, 2004.

[38] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.